Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Науково-навчальний інститут фізико-технічний інститут
Кафедра інформаційної безпеки

Лабораторна робота №3

з дисципліни: «Блокчейн та децентралізовані системи»

Тема: «Дослідження безпечної реалізації та експлуатації децентралізованих додатків.»

Виконали студенти групи ФБ-52мп: Бондаренко Я.В.
Пешкова М.Р.
Чуб М.В.

Перевірила: Селюх П.В.

Київ - 2026 рік

Мета роботи: отримання навичок роботи із децентралізованими додатками та оцінка безпеки інформації при їх функціонуванні.

Завдання: дослідження вимог OWASP (безпека web-додатків) та складання аналогічних вимог для обраної системи децентралізованих додатків.

Хід роботи

Частина 1. Еволюція методології OWASP у контексті сучасних загроз

На сучасному етапі розвитку IT-індустрії спостерігається фундаментальна трансформація архітектурних підходів: перехід від класичних централізованих моделей до розподілених децентралізованих систем, що тягне за собою радикальну зміну парадигми забезпечення кіберстійкості. Протягом тривалого часу методологія OWASP слугувала надійним фундаментом для мінімізації ризиків у традиційному веб-сегменті, надаючи галузеві стандарти та інструментарій захисту. Однак стрімке поширення децентралізованих додатків (dApps) на базі блокчейн-технологій створило нові вектори атак, які неможливо повноцінно нівелювати лише адаптацією існуючих практик. Це зумовлює гостру необхідність у впровадженні профільних нормативних баз, зокрема Стандарту верифікації безпеки смарт-контрактів (SCSVS).

У межах даної роботи проведено комплексний аналіз застосовності вимог OWASP Top 10 та ASVS до архітектури dApps. На основі отриманих результатів запропоновано модифіковану систему вимог для децентралізованих екосистем, яка враховує:

- Технологічні особливості та відмінності середовищ виконання Ethereum (EVM) та Solana (SVM).
- Актуальний ландшафт загроз, характерний для періоду 2025–2026 років.

Проект відкритої безпеки веб-додатків (OWASP) протягом десятиліть формував стандарти захисту, надаючи розробникам інструменти для мінімізації ризиків у веб-середовищах. Аналіз останньої редакції OWASP Top 10 (2025) демонструє стратегічний перехід від фокусу на конкретних технічних вразливостях до системних недоліків архітектурного проектування та контролю доступу. Категорія «Порушення контролю доступу» (Broken Access Control) посідає перше місце, охоплюючи значну частину протестованих додатків, що свідчить про критичну важливість ідентифікації та авторизації в розподілених системах. У 2026 році цей тренд лише посилюється через домінування API-центричних архітектур та інтеграцію агентних систем штучного інтелекту, де помилки в об'єктно-орієнтованій авторизації стають головним вектором атак.

Динаміка категорій OWASP Top 10 у періоді 2021-2025

Категорія (2025)	**Назва**	**Зміна позиції**	**Ключові аспекти та CWE**
A01:2025	Broken Access Control	Утримує 1	Консолідація SSRF; фокус на BOLA та витоку даних.
A02:2025	Security Misconfiguration	Підйом з 5 на 2	Неправильні налаштування хмар, дефолтні паролі.
A03:2025	Software Supply Chain Failures	Нова (експансія A06:2021)	Вразливості в залежностях, CI/CD та пайплайнах.
A04:2025	Cryptographic Failures	Падіння з 2 на 4	Слабкі алгоритми, неправильне управління ключами.

A05:2025	Injection	Падіння з 3 на 5	SQL/NoSQL ін'єкції, XSS, промпт-ін'єкції в LLM.
A06:2025	Insecure Design	Падіння з 4 на 6	Відсутність моделювання загроз, слабка архітектура.
A07:2025	Authentication Failures	Утримує 7	Слабкі потоки скидання паролів, відсутність MFA.
A08:2025	Software/Data Integrity Failures	Утримує 8	Неперевірені оновлення, шкідлива десеріалізація.
A09:2025	Logging & Alerting Failures	Утримує 9	Відсутність моніторингу, затримка реакції на інцидент.
A10:2025	Mishandling of Exceptional Conditions	Нова категорія	Небезпечна обробка помилок, стани "fail-open".

Категорія A01:2025 - Порушення контролю доступу

У контексті децентралізованих додатків контроль доступу трансформується з управління сесіями на сервері в управління дозволами на рівні смарт-контрактів та підписів гаманців. Традиційні вразливості, такі як Insecure Direct Object Reference (IDOR), у світі Web3 перетворюються на можливість виклику адміністративних функцій контракту неавторизованими користувачами, якщо не реалізовано належні модифікатори, такі як onlyOwner або системи ролей (RBAC). Статистика вказує, що 3.73% додатків мають вразливості, пов'язані з 40 різними CWE у цій категорії. Для dApps це означає

не лише ризик витоку інформації, а й повну втрату активів через несанкціоноване зняття коштів або зміну власника контракту.

Категорія A02:2025 - Неправильна конфігурація безпеки

Зростання цієї категорії до другої позиції обумовлене ускладненням стеків розробки. У децентралізованому середовищі це стосується не лише веб-серверів, а й конфігурації RPC-вузлів, оракулів та параметрів ініціалізації смарт-контрактів. Прикладом є використання дефолтних значень для лімітів проковзування (slippage) або неправильне налаштування проксі-контрактів, що дозволяє зловмисникам ініціалізувати логіку контракту повторно.

Категорія A03:2025 - Збої в ланцюжку постачання програмного забезпечення

Ця категорія є критично важливою для сучасних dApps, які часто покладаються на сотні сторонніх бібліотек (OpenZeppelin, Anchor, web3.js). Атаки типу "dependency confusion" або компрометація пакетів через npm (як у випадках з *Shai Hulud* або *React-Native-Aria*) демонструють, що зловмисники можуть впровадити шкідливий код, який підміняє адресу отримувача транзакції безпосередньо у фронтенді додатка. Для dApps вимога безпечного ланцюжка постачання включає верифікацію не лише власного коду, а й усього транзитивного графа залежностей та інструментів збірки.

Частина 2. Стандарт верифікації безпеки додатків (ASVS) як фундамент для dApps

OWASP Application Security Verification Standard (ASVS) версії 4.0.3 та 5.0 надає деталізовану структуру для оцінки безпеки, яка виходить далеко за межі базових чеклистів. Стандарт визначає три рівні верифікації, кожен з яких відповідає певній критичності додатка та апетиту до ризику.

Рівні верифікації ASVS

Рівень 1 (Opportunistic) орієнтований на додатки з низьким ризиком. Верифікація може бути проведена методами "чорної скриньки" без доступу до коду. Для dApps цей рівень є недостатнім, оскільки публічна природа блокчейну дозволяє зловмисникам бачити весь код і стан системи.

Рівень 2 (Standard) рекомендований для більшості комерційних додатків, включаючи фінансові платформи. Вимагає доступу до коду та документації. На цьому рівні перевіряється архітектура, аутентифікація та обробка даних.

Рівень 3 (Advanced) призначений для критичної інфраструктури та систем з високою цінністю активів. Більшість протоколів DeFi повинні прагнути до відповідності цьому рівню, оскільки будь-яка помилка веде до незворотних фінансових втрат.

Порівняння вимог ASVS для рівнів 2 та 3 у контексті dApps

Розділ ASVS	**Рівень 2 (Standard)**	**Рівень 3 (Advanced)**	**Застосування в dApps**
V1: Architecture	Аналіз загроз для основних функцій.	Повне моделювання загроз, формальна верифікація.	Проектування логіки смарт-контрактів та мостів.
V4: Access Control	Перевірка дозволів на рівні функцій.	Ешелонована оборона, захист метаданих.	Використання систем ролей та мультипідписів.
V6: Cryptography	Використання стандартних алгоритмів.	Апаратний захист ключів (HSM/TPM), ротація.	Управління приватними ключами адміністраторів.

V11: Business Logic	Перевірка цілісності робочих процесів.	Захист від автоматизованих атак та анти-фрод.	Стійкість до Flash Loan та маніпуляцій оракулами.

Частина 3. Спеціалізовані вимоги для децентралізованих додатків: SCSVS v2.0

Оскільки dApps мають унікальну архітектуру, де бекенд замінено смарт-контрактами, виникла потреба у стандарті Smart Contract Security Verification Standard (SCSVS). Цей стандарт пропонує аналогічну ASVS структуру, але сфокусовану на специфіці блокчейну.

Розділ G1: Архітектура та моделювання загроз

Безпека починається з ідентифікації всіх пов'язаних сутностей та аналізу меж довіри. Згідно з SCSVS v2.0, критичною вимогою є забезпечення можливості оновлення (Upgradeability) через проксі-патерни, але з обов'язковим захистом функцій оновлення за допомогою мультипідписів та таймлоків (Time-locks). Це дозволяє спільноті відреагувати на потенційно шкідливі зміни до їх впровадження.

Розділ G5: Контроль доступу та автентифікація

У dApps автентифікація базується на підписах криптографічних ключів. Вимоги безпеки включають:

- Використання стандартизованих протоколів підключення (наприклад, WalletConnect) для зашифрованого зв'язку між фронтендом та гаманцем.
- Запобігання атакам через "передбачувані повідомлення для підпису". Якщо dApp використовує статичне повідомлення, зловмисник може

використати фішинг для отримання підпису та імітації користувача (Replay Attack).

- Сувора перевірка підписанта (Signer) у кожній привілейованій функції. У Solana це вимагає обов'язкової перевірки поля is_signer.

Розділ G7: Арифметика та логіка

Блокчейн-системи оперують цілими числами з фіксованою точністю. Вимоги безпеки включають:

- Захист від переповнення (Overflow) та заниження (Underflow), і хоча в Solidity 0.8+ це вбудовано, розробники на Rust/Solana повинні використовувати методи типу checked_add або checked_sub.
- Захист від маніпуляцій оракулами - використання спотових цін з DEX є помилкою. Вимога полягає у впровадженні децентралізованих оракулів (Chainlink, Pyth) з використанням середньозваженої за часом ціни (TWAP).[1]

$$P_{TWAP} = \frac{\sum_{i-1}^{n} P_i \cdot \Delta t_i}{\sum_{i-1}^{n} \Delta t_i}$$

Математична репрезентація стійкості до маніпуляцій базується на вартості зміни ціни. Якщо протокол дозволяє змінити ціну всередині однієї транзакції без витрат, він вважається вразливим.

Частина 4. Порівняльний аналіз безпеки Ethereum (EVM) та Solana (SVM)

Розуміння архітектурних розбіжностей є ключовим для формування вимог безпеки. *Ethereum* використовує послідовну модель виконання, тоді як *Solana* - паралельну (Sealevel), що створює різні класи загроз.

Технічне порівняння вразливостей та вимог захисту

Параметр порівняння	**Ethereum (EVM)**	**Solana (SVM)**	**Вимога безпеки (SCSVS)**
Модель виконання	Послідовна	Паралельна	Захист від Race Conditions (G1)
Критичний ризик	Reentrancy (повторний вхід)	Account Substitution (підміна акаунта)	Валідація вхідних даних (G5)
Керування станом	Внутрішній стан контракту	Зовнішні акаунти даних	Перевірка власника акаунта (G5)
Мова програмування	Solidity, Vyper	Rust, C	Безпечне кодування (G11)
Захисні механізми	ReentrancyGuard, CEI патерн	Anchor framework, PDA верифікація	Використання перевірених фреймворків

Патерн Checks-Effects-Interactions (CEI) в Ethereum

Для боротьби з атаками повторного входу (Reentrancy), які стали причиною злому The DAO, обов'язковою вимогою є дотримання наступного порядку операцій:

1. **Checks:** Виконання всіх перевірок (require, assert).
2. **Effects:** Зміна внутрішнього стану (наприклад, балансу користувача).
3. **Interactions:** Зовнішні виклики або надсилання коштів.

Валідація акаунтів у Solana

Оскільки програми в Solana є stateless, вони зобов'язані валідувати кожен акаунт, що передається як аргумент. Відсутність перевірки того, чи належить акаунт очікуваній програмі-власнику, дозволяє зловмиснику підставити фейковий акаунт з маніпулятивними даними.

Частина 5. Аналіз інцидентів та Learned Lessons за 2025–2026 роки

Реальні випадки зломів демонструють, що навіть складні системи захисту можуть бути обійдені через комбіновані атаки, що поєднують технічні вразливості та соціальну інженерію.

Злом Drift Protocol (Квітень 2026) — $285 млн

Це найбільший злом 2026 року на Solana. Атака була здійснена через компрометацію Security Council та маніпуляцію системою оракулів.

Зловмисники (ймовірно пов'язані з КНДР) протягом місяців входили в довіру, створюючи "білі" стратегії в екосистемі. Вони використали функцію "durable nonces" для отримання підписів на транзакції, які були виконані пізніше для захоплення адмін-прав.

Таким чином вимоги безпеки повинні включати не лише аудит коду, а й аудит операційних процесів управління ключами та децентралізацію повноважень.

Інцидент Stream Finance та deUSD (Листопад 2025) — $93 млн

Інцидент демонструє ризики "економічної архітектури". Протокол використовував рекурсивні цикли ліквідності (looping), де один стейблкоїн забезпечувався іншим, що створювало ефект "фінансового вічного двигуна".

Втрата активів зовнішнім менеджером призвела до депегу (втрати прив'язки) токена deUSD, що за принципом доміно обвалило всю екосистему.

Таким чином вимоги SCSVS G4 (Business Logic) повинні включати стрес-тестування економічних моделей та обмеження концентрації активів.

Частина 6. Методологія аудиту та інструменти верифікації

У 2025-2026 роках процес верифікації безпеки став багатошаровим, поєднуючи автоматизований аналіз та експертну перевірку.

Етапи життєвого циклу безпеки (S-SDLC)

1. Етап проектування включає моделювання загроз за методологією STRIDE та аналіз інваріантів.

2. При розробці використовуються безпечні бібліотеки (OpenZeppelin) та виконується лінтинг коду.

3. Тестування:

- Статичний аналіз: Використання Slither (EVM) або Aderyn (Rust) для виявлення патернів небезпечного коду.
- Символьне виконання: Mythril для аналізу байт-коду та пошуку глибоких вразливостей логіки.
- Fuzzing: Використання Echidna або Trident для генерації мільйонів випадкових вхідних даних з метою порушення інваріантів (наприклад, можливість зняти більше коштів, ніж є на балансі).

4. Обов'язкова перевірка сторонніми фірмами (Trail of Bits, CertiK) – аудит.

5. Під час експлуатації необхідний моніторинг у реальному часі (Forta, Skynet) та впровадження "автоматичних вимикачів" (Circuit Breakers).

Ефективність інструментів безпеки смарт-контрактів у 2026 році

Інструмент	Тип аналізу	Покриття загрози	Особливість
Slither	Статичний	EVM	Виявляє ~70% типових помилок за секунди.
Mythril	Символьне виконання	EVM	Знаходить складні ланцюжки атак у байт-коді.
Trident	Фаззінг (MGF)	SVM	Спеціалізований для Solana, знаходить Account Confusion.
Echidna	Фаззінг	EVM	Найкращий для перевірки математичних інваріантів.
MIESC	Оркестрація	Multi	Об'єднує 35 модулів аналізу для високого Recall.

Частина 7. Формування уніфікованих вимог безпеки для обраної системи (dApp)

На основі проведеного дослідження вимог OWASP та SCSVS, можна сформулювати наступний перелік вимог для розробки та експлуатації безпечного децентралізованого додатка.

1. *Вимоги до архітектури (V1 / G1)*

- A-1.1: Проведення формального моделювання загроз (Threat Modeling) перед написанням коду. Кожне проектне рішення має бути обґрунтоване через призму безпеки.

- A-1.2: Реалізація механізму оновлення (Upgradeability) виключно через мультисиг-гаманці (мінімум 3 з 5 підписів) з таймлоком не менше 48 годин.
- A-1.3: Чіткий розподіл прав доступу. Адміністративні функції не повинні бути доступні через один і той самий акаунт, що використовується для щоденних операцій.

2. Вимоги до контролю доступу та ідентифікації (V4 / G5)

- AC-2.1: Заборона використання tx.origin для авторизації. Вся логіка повинна базуватися на msg.sender (для EVM) або перевірці is_signer (для SVM).
- AC-2.2: Захист фронтенду від XSS атак. Обов'язкове впровадження Content Security Policy (CSP) та використання SRI-хешів для всіх зовнішніх бібліотек (web3.js, ethers.js).
- AC-2.3: Перевірка прав у кожній функції (Function-level access control). Смарт-контракт повинен мати "fail-close" логіку - за замовчуванням доступ заборонено.

3. Вимоги до арифметики та бізнес-логіки (V11 / G7, G4)

- BL-3.1: Використання безпечних арифметичних операцій. Для Rust обов'язково застосовувати checked_ методи. Для Solidity версії < 0.8 обов'язково використовувати SafeMath.
- BL-3.2: Інтеграція децентралізованих оракулів (Chainlink/Pyth). Заборона використання спотових цін з DEX. Обов'язкова перевірка оракула на "свіжість" даних (staleness check).
- BL-3.3: Реалізація лімітів (Circuit Breakers). Автоматичне призупинення виведення коштів, якщо обсяг транзакцій перевищує встановлений поріг за короткий проміжок часу.

4. Вимоги до інфраструктури та експлуатації (V13 / G10, G6)

- INF-4.1: Оптимізація використання газу для запобігання DoS атак. Уникнення циклів по масивах невідомої довжини, які можуть призвести до перевищення ліміту газу в блоці.
- INF-4.2: Використання декількох RPC-провайдерів (failover strategy). Додаток не повинен залежати від працездатності одного вузла (Infura/Alchemy).
- INF-4.3: Постійний моніторинг та алертінг. Налаштування сповіщень про критичні події в контракті (зміна власника, великі зняття коштів).

Висновок: під час виконання лабораторної роботи було проведене дослідження безпечної реалізації та експлуатації децентралізованих додатків, яке підтверджує, що традиційні стандарти OWASP та ASVS залишаються критично важливим фундаментом, але вимагають суттєвого розширення через SCSVS для врахування специфіки блокчейну. Основні виклики безпеки 2026 року лежать у площині складних взаємодій між протоколами (Composability Risks) та управлінням доступом у децентралізованих структурах.

Для побудови стійкої екосистеми необхідно:

1. Дотримуватися ешелонованої оборони на всіх рівнях: від безпечного кодування смарт-контрактів до захисту фронтенд-інфраструктури.
2. Інтегрувати безпеку в кожен етап життєвого циклу розробки (Shift-Left), використовуючи сучасні методи фаззінгу та символьного виконання.
3. Забезпечувати прозорість та підзвітність через публічні аудити та системи моніторингу в реальному часі.

Тільки через синтез перевірених часом методологій AppSec та новітніх блокчейн-технологій можливо забезпечити надійне функціонування децентралізованих фінансових систем та захистити активи користувачів у ворожому цифровому середовищі.